SIGH — Acta de Cierre del Proyecto

Proyecto: SIGH (Sistema Integrado de Gestión de Horarios) | Asignatura: Bases de Datos 1
Programa: Ingeniería de Sistemas | Universidad El Bosque | Docente: Ing. Christian Felipe Duarte
Fecha de Cierre: 2 de noviembre de 2026 | Versión 1.0 Final

1. Propósito y Justificación del Cierre

El presente documento formaliza el cierre exitoso del proyecto Sistema Integrado de Gestión de Horarios (SIGH), habiendo dado cumplimiento al 100% de los entregables técnicos, de diseño, modelado, infraestructura, desarrollo y gestión establecidos en el enunciado de la asignatura Bases de Datos 1.

La solución entregada implementa un sistema relacional en PostgreSQL alojado en la nube de Amazon Web Services (AWS EC2), en el cual el motor de base de datos hace cumplir autónomamente las reglas de negocio a través de restricciones declarativas, 6 disparadores (triggers), políticas de seguridad por fila (RLS) y segregación de permisos en cuatro roles de acceso, complementado por una API REST en Spring Boot y una interfaz web SPA en Angular.

2. Balance de Cumplimiento de Entregables

• Fase 1 (Modelado y Requisitos): Propuesta, Supuestos, Requisitos, MER Chen, MER Extendido, MER Crow's Foot, Modelo Relacional, Normalización (1FN-3FN), Diccionario (20 tablas), Data Modeler y 50 ejercicios de Álgebra Relacional.
• Fase 2 (Infraestructura y RDBMS): Selección de SO Ubuntu Server LTS, Selección de RDBMS PostgreSQL 18.6, despliegue en 2 instancias EC2 (Web/API y Base de Datos privada), Nginx como proxy inverso y túnel SSH.
• Fase 2 (Capa Física y Reglas): Scripts 01 a 07 en PostgreSQL con 41 pruebas automáticas de reglas de negocio.
• Fase 2 (Software): API Spring Boot (Java 17, 69 endpoints, 4 pools RBAC) y Frontend Angular 21 (Material Design, JWT).
• Gestión: Acta de Constitución (con Presupuesto e Interesados), EDT/WBS, Cronograma, Plan de Pruebas y Acta de Cierre.

3. Cumplimiento de Requerimientos No Funcionales (RNF)

• RNF-01 (Integridad en BD): 22 reglas de negocio gobernadas por triggers y restricciones en PostgreSQL.
• RNF-04 (Seguridad por Roles): Conexión mediante 4 pools independientes (app_*); prohibición de superusuario.
• RNF-05 (Seguridad por Fila): Docentes restringidos exclusivamente a su disponibilidad mediante políticas RLS (RN-19).
• RNF-06 (Bitácora Inmutable): Trazabilidad de operaciones mediante triggers y set_config('app.id_usuario').
• RNF-08 (Disponibilidad y Salud): Monitoreo de estado en /api/echo y operación dentro de la Capa Gratuita de AWS.

4. Balance Financiero y Lecciones Aprendidas

• Ejecución Presupuestal: Costo financiero directo de $0 COP mediante el uso óptimo de la Capa Gratuita de AWS (t3.micro) y componentes de software libre (PostgreSQL, Ubuntu, OpenJDK 17, Angular, Nginx).
• Lección Clave: El gobierno de datos en el motor RDBMS y el aislamiento de instancias en subredes privadas garantizan máxima seguridad y estabilidad con cero sobrecostos.

5. Firmas de Aprobación

Ángel Daniel Aya Páez
Gerente de Proyecto / Pruebas

Julián Felipe López Narváez
Capa de Datos / Modelado

David Alejandro Ordaz Luque
Infraestructura / Software